

AWS Deployment Architecture

To deploy this MERN stack Mini Expense Tracker efficiently, I use a managed-services approach.

- **Frontend (React App): Amazon S3 + CloudFront**
 - Build React app into static HTML/CSS/JS files and store them in an **Amazon S3** bucket. As S3 service do not require manage it by hand.
 - Place **Amazon CloudFront** (a Content Delivery Network) in front of S3. It caches your frontend on servers globally so the UI loads instantly, and it provides a free SSL certificate for secure HTTPS connections.
- **Backend (Node.js/Express): AWS Elastic Beanstalk**
 - Deploy your server .js to **Elastic Beanstalk**. This is a Platform-as-a-Service (PaaS) that automatically sets up the necessary EC2 virtual servers to run your code.
 - It automatically provisions an **Application Load Balancer (ALB)** to manage incoming API requests and distribute them across your servers.
- **Database (MongoDB): MongoDB Atlas**
 - Use **MongoDB Atlas** and select the exact same AWS Region as your Elastic Beanstalk servers (e.g., us-east-1). Keeping them in the same region ensures near-zero latency when your backend talks to your database.

II. Overall Working Data Flow

1. **UI Delivery:** The user navigates to your domain. CloudFront intercepts the request and instantly serves the cached React application from the S3 bucket.
2. **User Action:** The user fills out the "Add Expense" form and clicks submit.
3. **API Request:** React makes an HTTP POST request to your backend domain.
4. **Load Balancing:** The AWS Application Load Balancer receives the request and forwards it to a healthy Node.js server.
5. **Database Execution:** Express validates the request body using Mongoose. It opens a connection to MongoDB Atlas, saves the document, and returns a 201 status code.
6. **UI Reaction:** The React app receives the success response, clears the form, and triggers parallel GET requests to instantly update the list and the summary aggregation.

III. Scalability & System Design Concepts

You can use these concepts to directly answer the "Quick Discussion" questions required by your interviewer:

- **Database Indexing (Performance):** If the database grows, querying becomes slow. You should place an index on the category field and the createdAt field. This optimizes your GET /api/expenses?category=food route and your .sort({ createdAt: -1 }) command, changing a slow full-table scan into a lightning-fast lookup.
- **Pagination (Data Management):** If a user has 100,000 expenses, sending them all at once will crash the browser and max out backend memory. You would update the GET endpoint to accept ?page=1&limit=20 queries, using MongoDB's .skip() and .limit() to only return small chunks of data at a time.
- **Horizontal Scaling & Statelessness:** By storing your database in Atlas and not saving any files locally on the Node server, your backend is "stateless." If traffic spikes, the AWS Load Balancer can safely spin up 5 identical Node.js servers, and any server can handle any request perfectly.

IV. System Architecture Diagram